LANChat-H: Design, Implementation and Evaluation of a Dependency-Light Secure LAN Chat System with End-to-End Encryption

1st Harish Goud B
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
harishgoudbit@cbit.ac.in

2nd Revanth Reddy Lingala
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
revanth252017@gmail.com

3rd Saarthak Manocha
Dept. of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, India
saarthakmanocha10@gmail.com

Abstract*—Small teams and classrooms often want a private chat that stays on their own network, but the usual options are either heavy (federated protocols, message brokers, web stacks) or simple demonstrations with no persistence, no identity and no protection from anyone on the network or from the person running the server. We present LANChat-H, a chat system for local networks whose core uses only the Python standard library. It combines a length-prefixed TCP protocol with a plain-text fallback for older clients, a bounded outbound queue per client that isolates slow receivers, TLS with trust-on-first-use certificate pinning, memory-hard credentials with lockout and rate limits, server-checked roles, SQLite persistence, ticket-authorised file transfer and a hash-chained audit log. On top of these it adds end-to-end encryption, built from audited primitives (X25519, Ed25519, ChaCha20-Poly1305), for private messages, for files and, optionally, for the group room, so that the server stores only ciphertext. The room uses one-way key chains per sender and per epoch: a late joiner reads only what follows, and a removed member never receives the new keys. On a single laptop over loopback, median delivery latency stayed between 2.7 and 5.8 ms for up to 100 receivers; when one receiver stopped reading, a sequential-relay baseline stopped delivering to everyone after about 232 messages whereas LANChat-H delivered all 6000 of 6000; encrypting a message costs about 0.3 ms and a 20 MB file about 31 ms; and a test that records every frame the server handles, and scans them together with its database and stored files, finds none of the secrets of encrypted conversations. The roughly 10,500 lines are covered by 452 automated tests and 370 scripted checks that drive the real window. We state the limits plainly: no forward secrecy, metadata visible to the server, measurements from one machine, and no asynchronous baseline.

***Index Terms*—LAN chat, end-to-end encryption, system design, message framing, head-of-line blocking, trust on first use, group messaging, audit log, Python**

I. Introduction

Small teams, classrooms and labs often want somewhere to talk and share files that stays on their own network. The mainstream answers are heavy: federated protocols such as XMPP, brokers such as MQTT, or web stacks with databases, containers and accounts held by third parties. The lightweight answers are the opposite: a few dozen lines of sockets that broadcast text, with no identity, no history, no protection from anyone on the network and none from whoever runs the server. Between the two there is little that a student, a teacher or a small office can read in an afternoon, start with one command, and still trust with private conversations.

Filling that gap exposes four problems that look separate but interact. First, TCP offers a byte stream, so a chat protocol must define its own message boundaries [2], [7], and that definition has to survive new features and older clients. Second, a server with many long-lived connections must not let one slow or stalled client delay the others, a head-of-line problem that sequential broadcast has by construction. Third, a local network is not a trust boundary: any host can connect, sniff and impersonate, so the system needs transport security, identity, throttling and an audit trail. Fourth, the person who runs the server should not automatically be able to read private conversations, which calls for end-to-end encryption in a system whose server also relays, stores, searches and moderates messages.

This paper presents LANChat-H, a chat system for local networks that addresses all four in one small code base whose core uses only the Python standard library [32], [33]. It began as a minimal threaded relay and grew through a sequence of milestones, each leaving earlier behaviour and the terminal client working (Section IV-H). The contributions are:

1) a protocol and server design that keeps framing simple and compatible: length-prefixed frames carrying plain text or versioned JSON events, a renderer that lets text-only clients keep working as features are added, and a bounded outbound queue with a writer thread per client that isolates slow receivers;
2) a layered security design for a trusted-network deployment: TLS with trust-on-first-use pinning, memory-hard credentials with lockout and rate limits, roles checked on the server, ticket-authorised file transfer with hash verification, and a hash-chained audit log;
3) an end-to-end encryption layer for a server that also stores and relays: private messages and files, and an optional group room built on one-way per-sender key chains per epoch (late joiners read only what follows, removed members never receive the new keys), with key rotation, recovery and an app lock, all from audited

primitives;
4) an implementation of about 10,500 lines with one optional third-party package, covered by 452 automated tests and 370 scripted checks of the real window, including a test that scans every frame the server handles for the secrets of encrypted conversations; and
5) a measured evaluation on one machine, with the limits of the design and of the measurements stated.

Section II places the work among related systems. Section III states the goals and the threat model. Section IV describes the design, and Section V the implementation. Section VI evaluates it, Section VII discusses lessons and limits, and Section VIII concludes.

II. Background and Related Work

Transport and framing. TCP provides a reliable, ordered, full-duplex byte stream and nothing more [2], [1], [4], [5], [8], [9]: one `send()` need not correspond to one `recv()`, and functions that only the application can complete, such as message boundaries, cannot be delegated to the lower layer [7], [6]. Delimiter framing, as in the CRLF lines of IRC [18], is readable but needs an escaping rule; XML streams as in XMPP still need an outer mechanism to find where a unit ends [20]; WebSocket and MQTT carry lengths in binary headers [22], [23]. TCP keep-alive is optional and off by default [3], so noticing an unplugged peer needs application-level heartbeats. LANChat-H uses a four-byte network-order length [34], [10], which is binary-safe and exact at the price of validating the announced length.

Serving many clients. Thread-per-client gives each connection a sequential control flow that is easy to read and debug; event loops multiplex many connections on few threads but turn each into a state machine [17], [16]; staged designs such as SEDA join event-driven stages with bounded queues [12]; and the threads-versus-events debate is old and unresolved [13], [14], [15], [11]. Large systems isolate slow consumers with per-connection queues or non-blocking writes, but teaching-scale servers usually broadcast sequentially, so one stalled receiver delays everyone after it. LANChat-H keeps threads for readability and adds the isolation (Section IV-C); it does not claim that threads are the right choice at scale.

Messaging systems and secure messaging. IRC, XMPP, MQTT and WebSocket each solve interoperability and deployment problems (federation, brokers, browsers) that a single-server LAN chat does not have [18], [19], [20], [21], [23], [22]; Table I compares them with LANChat-H. TLS protects the channel [28], but the server still sees plaintext. End-to-end designs show how to hide content from a relaying server: the Double Ratchet [31], [30], X3DH for asynchronous contact set-up [47], and Messaging Layer Security for efficient groups [46], built on X25519, Ed25519, HKDF and ChaCha20-Poly1305 [42], [43], [45], [44]; Unger *et al.* compare the designs [29]. LANChat-H reuses the primitives and the sealed-key and sender-key ideas, not these full protocols.

Accounts, trust, audit, discovery and transfer. Authentication guidance recommends salted memory-hard hashing, checks against common passwords and throttling of repeated failures [48], [41]. SSH's trust-on-first-use host-key model is the precedent for self-signed certificates [38]. Hash-chained logs make alteration detectable, with a separate commitment needed to detect truncation [49], [50]. LAN services are normally found with IP multicast [24] and multicast DNS [39], [40], while small programs use a UDP broadcast, and FTP separated control and data connections [37], the pattern LANChat-H uses so that a large transfer never blocks the chat. Publish–subscribe and reliable group communication offer delivery guarantees far beyond what a single-server chat provides [25], [26], [27].

Each of these works treats one concern well. What is missing is a small artifact in which framing, isolation of slow clients, trust, persistence and end-to-end encryption can be read, tested and measured together. That is the niche LANChat-H occupies. It is a systems contribution, not a new protocol or cipher: the novelty is in how the pieces are combined and checked.

III. Design Goals and Threat Model

Goals. We set seven goals, each tied to something the evaluation checks.

- G1 *Small and runnable.* Standard library for the core, one folder, one command, modules small enough to read.
- G2 *Compatible growth.* New features must not break clients that predate them.
- G3 *Isolation.* One slow, stalled or hostile client must not delay or crash the others.
- G4 *Safe on a hostile network.* Encryption in transit, a way to recognise the server, real accounts, throttling, and no file or name that can harm a recipient or the host.
- G5 *Private from the host where it matters.* Private conversations, their files and (optionally) the group room must be unreadable to the server, with keys that survive a lost computer.
- G6 *Accountable and operable.* Roles, an audit trail that shows tampering, administration tools, backups and upgrades that keep data.
- G7 *Verifiable.* Every claim above is backed by a test or a measurement that can be rerun.

Threat model. Table II lists whom we defend against. We assume the clients' computers are not compromised and that the person comparing security codes is who they say they are. We do not try to hide metadata, to defend a compromised endpoint, or to scale beyond tens of people.

IV. System Design

A. Architecture

Fig. 1 shows the structure. The server owns membership, persistence and policy; clients are thin. Both client kinds hold one TLS connection to a central relay, and file bytes use a second, short-lived connection authorised by a ticket from the first. A UDP responder lets clients find servers on the same network.

TABLE I
COMPARISON OF RELATED APPROACHES WITH LANCHAT-H

Approach	Transport	Framing	Topology	Security	Persistence	Key limitation for a small teaching system
IRC [18], [19]	TCP	CRLF lines, 512-byte limit	Server network	None in base spec	None	Text-only framing; limits fixed by the spec
XMPP [20], [21]	TCP	XML stream	Federated client–server	SASL and TLS	Via extensions	Heavy parser; large specification surface
MQTT 5.0 [23]	TCP (optionally TLS)	Binary header, variable length	Broker pub/sub	Optional; credentials in CONNECT	Sessions and QoS	Needs a broker; more machinery than a chat room needs
WebSocket [22]	TCP after HTTP upgrade	Length-carrying frames; client masking	Client–server	TLS via `wss`	None	Needs an HTTP handshake and a framework
Signal / Double Ratchet [31], [30]	Any	Not specified	Relayed	End-to-end	Server-dependent	Cryptographic design, not a transport or server design
FTP [37]	TCP, separate control and data connections	Commands as text lines; data as a raw stream	Client–server	None in base spec	Files only	Predates TLS; no chat, presence or history
Flash [11]	TCP (web)	HTTP	Client–server	N/A	N/A	Web workload; compares process, thread and event models
SEDA [12]	TCP (services)	Application-defined	Staged pipeline	N/A	N/A	Targets far larger loads than a classroom chat
Threads vs. events [13], [14], [16]	TCP	N/A	N/A	N/A	N/A	Position papers, not chat-specific
Reliable groups [26], [27]	Reliable channels	N/A	Process groups	N/A	Varies	Guarantees far beyond a demonstration system
LANChat-H (this work)	**TCP + TLS**	**4-byte length + text or JSON events**	**Central relay**	**TLS with pinning, scrypt, roles, audit, end-to-end encryption**	**SQLite**	**Single server; no forward secrecy; metadata visible to the server (Sec. VII)**

TABLE II
ADVERSARIES AND WHETHER THEY ARE DEFENDED AGAINST

Adversary	Outcome
Another host on the network	Defended: TLS, certificate pinning, lockout, rate limits
A member beyond their role	Defended: roles checked on the server; hostile files and names neutralised; replays and forged senders detected
The host or a thief of its disk	Partly: private messages, private files and the optional room are ciphertext; metadata and anything sent unencrypted are not
A server that lies about keys	Detected, not prevented: pinning and safety numbers
A stolen, unlocked computer	Not defended; auto-lock limits exposure

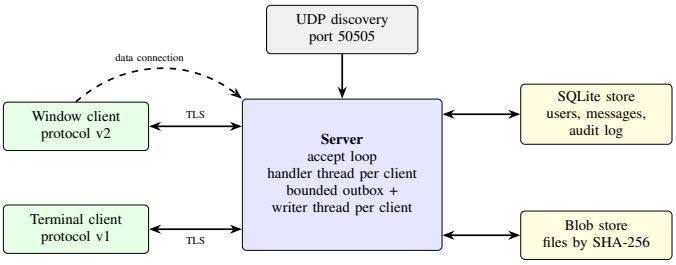


Fig. 1. Architecture of LANChat-H. Both client kinds hold one TLS connection to a central relay; file bytes use a second, short-lived data connection authorised by a ticket from the first.

B. Framing, Protocol Versions and Compatibility

Every message in both directions is a four-byte big-endian length followed by UTF-8 text, as in Listing 1; zero-length and over-long frames close the connection. Two protocol versions share this framing. Version 1 carries plain text lines and is what the terminal client speaks. Version 2 carries one JSON object per frame with a type field (message, edit, delete, reaction, typing, presence, read marker, pin, search, history page, upload, download, key and room events, and so on); a version-2 client announces itself with a JSON hello in the username step, and the server detects the version from the first character. For version-1 clients the server renders every event as the line they have always seen, and sends nothing for events a terminal cannot show, so features were added without breaking the older client (G2). Message text is limited to 3400 bytes so that the JSON envelope always fits one frame.

C. Concurrency and Delivery

Each connection has a handler thread that reads requests and a writer thread that is the only code writing to that socket. Delivering a message means placing it on every recipient's bounded outbound queue (default 100 entries) without waiting.

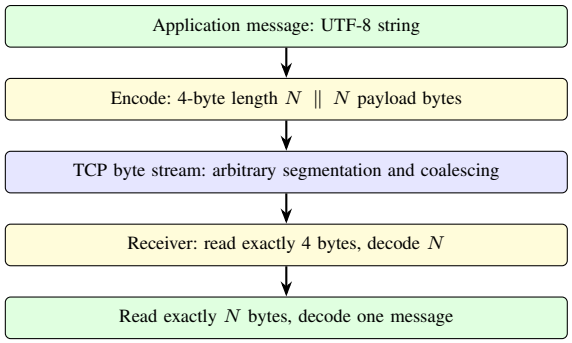


Fig. 2. Length-prefixed framing over a TCP byte stream.

```python
MAX_MESSAGE_BYTES = 4096

def send_message(conn, message):
    payload = message.encode("utf-8")
    if len(payload) > MAX_MESSAGE_BYTES:
        raise ValueError("message is too large")
    conn.sendall(struct.pack("!I", len(payload)) + payload)

def receive_message(conn):
    header = _receive_exactly(conn, 4)
    if header is None:
        return None             # peer closed
    (length,) = struct.unpack("!I", header)
    if length == 0 or length > MAX_MESSAGE_BYTES:
        raise ValueError("invalid message length")
    payload = _receive_exactly(conn, length)
    if payload is None:
        return None
    return payload.decode("utf-8")
```

Listing 1. Framing logic of the reference implementation (`protocol.py`, condensed).

If a queue fills, the client is not keeping up and is disconnected; it can reconnect and replay history from the database (G3). The welcome and history are written directly before the writer thread starts, so they arrive first and in order. Clients send a ping every ten seconds and the server drops a connection silent for thirty. A user name may be signed in once; a saved sign-in may take over a stale connection. Section VI measures the effect of the queues.

D. Persistence and Features

Messages, users, reactions, pins, read markers, sessions, public keys and the audit log live in SQLite, so history survives restarts; seven numbered migrations let an older database upgrade in place (G6). A joining client receives the last 50 messages and can page back 50 at a time. Search uses an FTS5 index and falls back to `LIKE` where FTS5 is absent. The window supports replies, reactions, edit and delete (deletion keeps a tombstone so quoted replies still resolve), typing indicators, presence, read receipts, mentions, pins and an administrator console. These features stress the protocol: history paging and search need replies that respect back-pressure, edits and deletes need ids that survive reconnection, and, as Section IV-G shows, search, pins and mentions need rethinking when the server cannot read the text.

E. Bulk Transfer

Pictures, documents and audio do not travel on the chat connection. The client sends an upload request with the name, size and SHA-256; the server checks role, type, size, quota and conversation and replies with a one-time ticket. The client opens a second TLS connection, presents the ticket, and streams raw bytes; the server writes them to a temporary file while hashing, enforces the declared size and a deadline, compares the digest, and only then moves the file into place under its hash and posts the message. Downloads mirror this and are verified by the receiver; a damaged file is never delivered, and a half-finished upload leaves nothing behind. Identical content is stored once, and a file is removed when the last message referring to it is deleted. File names are reduced to a safe base name and executable types are blocked (G4).

F. Transport and Account Security

Table III lists each threat with its control; the controls are in code and covered by tests (Section VI). Together they implement G4 and the non-confidentiality parts of G6; the confidentiality controls of G5 are described next.

G. End-to-End Encryption

End-to-end encryption is the part of the design that changes what the server is. Without it the server decrypts TLS, stores and relays plaintext and can search it; with it the server stores ciphertext and public keys, and everything that needs the words moves to the client.

Identities and trust. Each person has an X25519 key pair for key agreement and an Ed25519 pair for signing, bound by a self-signature [42], [43]. Secret keys are wrapped on disk with ChaCha20-Poly1305 [44] under a key derived by scrypt from a passphrase. A 24-character recovery code (about 119 bits) wraps a second copy that the server stores as an opaque backup; because the code is high entropy the server cannot usefully guess it, and a test confirms that neither the keys nor the code appear in the database file. The server keeps only public keys and cannot be trusted to report them honestly, so clients verify each bundle's signature, pin the first key seen for each contact, block sending when it changes until the person has accepted the new one, and offer a sixty-digit safety number computed from both identities for comparison in person or on a trusted call.

Private messages. The sender encrypts the text once under a random key with ChaCha20-Poly1305, seals that key separately for the recipient and for themselves (ephemeral and static X25519 through HKDF-SHA256 [45]), and signs the whole message with Ed25519, binding the sender and recipients into the associated data. The server checks only the shape and the two names, stores an empty body and the envelope, and relays it. Tampering, a wrong recipient or a forged sender fails to open, and the window flags a reused random message id as a possible replay. Encrypting for the sender as well as the recipient costs a few hundred bytes and lets people read their own history after a restart.

TABLE III
THREATS AND CONTROLS IN LANCHAT-H

Threat	Control
Reading or altering traffic	TLS (1.2 or newer) on chat and data connections. The host creates a self-signed certificate once.
Impersonating the server	Trust on first use: the client shows the certificate's SHA-256 fingerprint, pins the certificate after approval and refuses a different one later. Host name checking is off because LAN addresses change; the pinned certificate authenticates the server.
Stolen account database	Passwords stored only as salted scrypt hashes ($n = 2^{14}$, $r = 8$, $p = 1$, 32-byte output) with constant-time comparison.
Password guessing	Five wrong passwords lock a name from an address for 30 s, doubling to 15 min; one address failing for many names is locked after 20; a correct password clears the count. New passwords need six characters, must differ from the user name and common passwords; at most ten sign-ups per address per hour.
Flooding	Per-person token bucket (30 burst, 10 per second); 240 new connections per address per minute; frame, message and file size limits.
Privilege abuse	One permission table; every action checked on the server; the last admin cannot be demoted or disabled.
Password kept in memory	After sign-in the window keeps a random token (12 h), stored on the server only as a SHA-256 hash, revoked on leave, password change, disable or reset.
Hostile files	Names reduced to a base name; files stored under their SHA-256; executable types blocked; size and space limits enforced while streaming; hash checked on upload and on download; tickets are single-use, expire in 60 s and are limited per person.
Disputed history	Audit log of sign-ins, failures, lockouts, role changes, deletions of others' messages, uploads, resets and settings, each row hashed with its predecessor; a head marker detects truncation; message text is never logged.
The host reading chats and files	End-to-end encryption of private messages, files and (optionally) the group room: the server stores ciphertext and public keys only (Sec. V-G).
A fake key from the server	Self-signed key bundles, pinning on first sight, sending blocked after a change until accepted, sixty-digit safety numbers.
Replays and forged senders	Every encrypted message is signed and bound to its sender and recipients (or epoch and number); the window flags reused ids and numbers.
A removed member	A new key epoch for the room, whose keys the removed person never receives.
An unattended window	Auto-lock after an idle time covers the window and forgets the keys.

Files. Files are encrypted on the sender's computer in 64 KiB chunks under a random key; each nonce carries the chunk number and a last-chunk flag, so dropped, reordered or added chunks fail. The key, the real name and the type travel inside the encrypted message, so the server stores unreadable bytes under an opaque name and can apply only size and space limits; the receiving window, which alone knows the name, warns before opening a program-like file.

The group room. A sealed copy for every member does not scale to a room, so the room uses sender keys in the style of group messengers [46]. Each member holds a random chain key per *epoch*, derives the key for message n by n one-way steps, and gives every other member a sealed copy of the chain as it stands, so a member who joins later reads only what follows. The server stores the sealed copies (so they can be fetched again after a restart), enforces the current epoch and strictly increasing message numbers per sender, and starts a new epoch whenever someone is removed or an administrator asks; the removed person never receives the new chain. Mentions are named by the sender because the server cannot read the text, and `@everyone` stays administrator-only; pins, quoted replies and search use what the window has decrypted. An administrator switches the room on, after seeing which members have no keys, because people without keys and terminal clients can then neither read nor write.

Key lifecycle. Keys can be rotated (a new identity, with the old ones kept so earlier messages still open and the server keeping the envelopes sealed to them), restored from the recovery code on a new computer with every identity, or reset, which discards history. An idle window locks itself, covering the interface, hiding dialogs and forgetting the keys until the passphrase is entered.

What the server still learns. The server sees who talks to whom, when, the sizes of messages and files, reactions, who is mentioned, each person's role and the membership of the room; it can also withhold or delete messages, though it cannot read or forge them. Section VII lists the resulting limits.

H. Development Path

LANChat-H was built in the milestones of Table IV. Each ended with the whole test suite and the scripted window runs passing, and with the terminal client unchanged in behaviour (G2). The order matters: persistence and a versioned protocol came before the features that need message ids, security hardening before file sharing opened new attack surface, and end-to-end encryption last because it depends on identities, a key directory, the ticketed transfer path and the event schema all being stable.

V. IMPLEMENTATION

The core uses only the Python standard library [32], [33], [34], [35]: `sqlite3` and `hashlib` supply storage and scrypt [51], [52], and `tkinter` supplies the desktop window. Three third-party packages are optional: `cryptography` for end-to-end encryption, Pillow for some picture previews and `sounddevice` for voice recording; without the first the chat works over TLS only and the window says so (G1). In total the non-graphical code is about 4,800 lines in 21 modules, the window about 4,900 lines in seven modules, and the end-to-end encryption code about 900 lines in three modules. Table V maps the modules to the design. The tests start real servers on free ports, and scripted drivers open the real window and click

TABLE IV
MILESTONES OF THE IMPLEMENTATION

Milestone	What was added
Baseline	Threaded relay, four-byte framing, plaintext broadcast, three tests
Robustness	Bounded queue and writer per client, heartbeat, unique names, TLS with pinning, accounts
Persistence and roles	SQLite, migrations, history, admin/member/guest roles
Social features	JSON events with a text fallback; replies, reactions, edits, presence, read receipts, search, mentions, pins, history paging
Hardening	Lockout, rate limits, password rules, saved sign-ins, password change, data export
Media	Ticketed file transfer, inline pictures, audio, hash verification
Administration	Admin console and CLI, hash-chained audit log
E2EE 1: keys	Identities, key directory, recovery code, safety numbers, pinning
E2EE 2: messages	Encrypted private messages, replay and forgery checks, policy
E2EE 3: files	Chunked file encryption, opaque server storage
E2EE 4: room	Per-sender key chains, epochs, late joiners, removal re-key
E2EE 5: hardening	Key rotation, app lock, wire-scan test, documentation

through it; the benchmarks of Section VI are in the repository [53].

VI. EVALUATION

All measurements were taken on one machine: an Apple-silicon laptop (arm64) running macOS 27.0.1 with CPython 3.13.9, with the server and clients on the loopback interface. The source code, tests and scripts (`bench/`) are published [53], and the numbers below are what they printed. Because client and server share one machine, the results show the cost of the software, not of a real network; they are not claims about Wi-Fi or Ethernet performance.

A. Correctness and Security Tests

The repository holds 22 test modules that run 452 tests, each starting real servers on free local ports. They cover framing (round trip, closed connection, oversized and zero-length frames), the version-1 and version-2 protocols, the database and its migrations, search and paging, TLS and certificate pinning including a changed certificate, discovery, file transfer (round trips, hash mismatch, cut-off uploads, hostile names, expired and reused tickets, quota, concurrent uploads), lockout, rate limits, password rules, saved sign-ins, administrator actions, tamper detection in the audit chain, and the encryption code, which is checked against published test vectors for X25519 and Ed25519 [42], [43] and then against tampering, wrong recipients, forged senders, replays, damaged and reordered files, and the room's rules for late joiners, removal and message numbering. One test wraps the server's send and receive functions, records every frame the server handles, and scans those frames, the database file and the stored files for the secret text, file names, file contents and file keys of encrypted messages; a companion check shows that the same capture would reveal a plaintext message. A further eleven scripts drive the real window and the terminal client through 370 checks, including runs with every security default on. All of them pass.

B. Fan-Out Latency

One sender sent 100 messages of about 100 bytes at ten per second to N receivers, each a real client with its own connection; every message was also written to the SQLite database before delivery. Receivers ran in separate processes of at most 20, and latency is the arrival time minus the send time on the machine's monotonic clock. Table VI shows that every message reached every receiver. Median latency stayed between 2.7 and 5.8 ms and the 95th percentile below 9.5 ms without TLS; with TLS the figures for 25 and 100 receivers were 4.7 and 6.4 ms (median) and 7.5 and 11.3 ms (95th percentile).

C. Head-of-Line Blocking

The fan-out cost model predicts the effect of a stalled receiver. With sequential sends, recipient k receives the message after

$$D_k = \sum_{j=1}^{k} s_j, \qquad T_{\text{bcast}} = D_N, \tag{1}$$

where s_j is the time the blocking send to recipient j takes, and if recipient m stalls for an extra time W then

$$D_k \;\geq\; W + \sum_{j=1}^{k} s_j \qquad \text{for all } k \geq m. \tag{2}$$

With a bounded queue and writer thread the send to each recipient reduces to an enqueue with cost $q_j \ll W$, so D_k no longer depends on W for $k \neq m$.

To test this we compared LANChat-H with a minimal sequential relay written for the experiment: a thread per client that sends each incoming message to every other client in turn with a blocking `sendall`. Both use the same framing. Ten receivers read normally; one more connects first, shrinks its receive buffer to 4 KiB and never reads. A sender sends 600 messages of 2000 bytes at 50 per second. Table VII gives the range over four runs. Once the kernel buffers for the stalled connection were full, after 232 messages (about 460 kB), the sequential relay stopped delivering to every receiver: exactly 2320 deliveries (232 messages to each of ten receivers) arrived in every run, out of 5370 to 5400 expected, and the sender's own sends began to time out after 537 to 540 messages. With the stalled receiver present LANChat-H delivered 6000 of 6000 messages to the ten healthy receivers, and dropped the stalled connection once its queue filled.

The table also shows the price. In the absence of a stall the sequential relay is faster per message (median under 1 ms against 4 ms), because it does no more than forward bytes, whereas LANChat-H also persists each message, checks the sender's role and rate limit, and renders it for each protocol version. Isolation of slow receivers is therefore a trade of a few milliseconds of latency for the guarantee that one bad connection cannot stop the room.

TABLE V
IMPLEMENTATION MAPPING AND HONEST SCOPE

Component	Concept	Role in the system	Boundary / not provided
`protocol.py`, `events.py`	Record recovery; typed events	4-byte network-order length, exact-read loop, 4096-byte limit; JSON events with a type field, plus a renderer that turns any event into a plain line for old clients	No acknowledgements or sequence numbers on the wire
`server.py`, `server_admin.py`, `server_keys.py`, `server_room.py`	Accept, membership, fan-out, policy	Handler thread and bounded outbox with writer thread per client; handshake; history, search, pins; ticketed file transfer; admin and key-directory events	Single server; no failover; one thread per client
`store.py`, `accounts.py`, `permissions.py`	Persistence, credentials, roles	SQLite with six numbered migrations, FTS5 search with `LIKE` fallback; scrypt hashes; admin, member and guest roles checked on the server	One connection behind a lock; not a clustered database
`security.py`, `audit.py`	Abuse resistance; accountability	Lockout with backoff, token-bucket rate limits, connection and sign-up caps, password rules; hash-chained audit log	In-memory counters reset on restart
`storage.py`, `transfer.py`	Bulk transfer	Content-addressed blob store, name sanitising, blocked types, streaming SHA-256 verification	Files are not encrypted at rest on the host
`chat_core.py`, `certs.py`, `discovery.py`	Client handshake, TLS trust, discovery	Login, certificate pinning by SHA-256 fingerprint, self-signed certificate creation, UDP broadcast discovery	Discovery works within one broadcast domain
`e2ee.py`, `e2ee_group.py`, `keystore.py`	End-to-end encryption	Identities, passphrase-wrapped key file, recovery code, safety numbers, pinned contacts, sealed messages and files, the room's key chains	No forward secrecy; the server still sees metadata (Sec. VII)
`gui_*.py`, `client.py`, `chat.py`, `admin_cli.py`	Interface	Desktop window, terminal client, single launcher, offline admin tool	Terminal client lacks edits, reactions, search and pictures

TABLE VI
BROADCAST LATENCY VS. RECEIVERS (LOOPBACK, MS)

N	TLS	Median	p95	Max	Delivered
1	no	3.23	4.64	6.98	100 %
5	no	3.67	4.93	16.65	100 %
10	no	2.71	5.23	7.78	100 %
25	no	2.72	5.91	11.51	100 %
50	no	4.16	6.65	26.70	100 %
100	no	5.83	9.40	13.94	100 %
1	yes	2.88	4.69	6.00	100 %
25	yes	4.66	7.46	13.59	100 %
100	yes	6.44	11.32	21.33	100 %

TABLE VII
ONE STALLED RECEIVER, TEN HEALTHY ONES

Server	Stalled	Delivered	Median ms	p95 ms
Sequential relay	no	6000/6000	0.65–0.78	1.36–5.46
Sequential relay	yes	2320/5370–5400	0.61–0.85	1.38–13.03
LANChat-H	no	6000/6000	3.75–4.36	5.36–8.82
LANChat-H	yes	6000/6000	3.16–4.58	5.53–5.92

D. File Transfer Speed

Table VIII gives the median of three runs for uploading and then downloading a random file through the ticket and data-connection path, including hashing, the ticket exchange and verification. Throughput is in the hundreds of megabytes per second on loopback and TLS costs roughly a third for small files and less for large ones. These figures show that the design adds no obvious bottleneck; they say nothing about a wireless link, which would dominate in practice.

TABLE VIII
FILE TRANSFER THROUGHPUT (LOOPBACK, MB/S, MEDIAN OF 3)

File (MB)	TLS	Upload	Download
1	no	282	636
1	yes	171	303
5	no	409	539
5	yes	370	464
20	no	616	595
20	yes	394	461

E. Cost of Security Operations

Table IX lists the median time of single operations. A password check costs about 27 ms by design: it is the price of making offline guessing expensive. Encrypting a message for one person (a random key sealed twice and a signature) takes about 0.3 ms and decrypting it 0.26 ms, a room message under 0.3 ms, and a 20 MB file is encrypted in about 31 ms and decrypted in about 23 ms (roughly 650 and 850 MB/s). The cost of end-to-end encryption is therefore dominated by passphrase stretching and key management, not by the ciphers, and the server's own work is unchanged: it relays the same frames.

F. Confidentiality Check

The confidentiality claim of G5 is checked by observation rather than by argument. The test wraps the server's receive

TABLE IX
COST OF SINGLE OPERATIONS (MEDIAN OF 15 RUNS)

Operation	Time (ms)
Password check (scrypt $n = 2^{14}$)	27.3
Wrap secret keys with a passphrase (scrypt $n = 2^{15}$)	57.7
Generate an identity (X25519 and Ed25519)	0.05
Safety number (5200 SHA-512 rounds)	2.1
Encrypt a 200-byte message for one person	0.31
Decrypt it	0.26
Room message: encrypt / decrypt	0.17 / 0.26
Seal a chain key for one member	0.25
Walk a room chain forward 1000 steps	1.3
Encrypt / decrypt a 20 MB file	30.7 / 23.4

and send functions so that every frame it handles is recorded. It then has two people exchange a private message, a private edit, a room message and an encrypted file with a caption, and scans all recorded frames, the SQLite file and the stored files for the secret message texts, the real file name and type, the file contents, the file key and the people's secret keys. None appears; the traffic does contain the encrypted envelopes, so the capture is real. A companion check sends a plaintext message through the same capture and finds it, so the scan would notice a leak. Separate tests assert that the database holds an empty body and an opaque attachment name for encrypted messages, that neither a chain key nor a recovery code is stored, and that a removed member, a late joiner and a person without keys cannot open what they should not.

G. Threats to Validity

The experiments use one machine, one Python version and the loopback interface, so queueing in the network, wireless loss and operating-system differences are absent. Latency includes scheduling of separate Python processes and is not a pure network figure. The synthetic load is regular (fixed rate and size), not a trace of real conversations. The sequential relay is a minimal baseline written for the comparison, not a measurement of any other published system, and the claim that thread-per-client stops being adequate at some size has not been tested because no asynchronous variant was built. The tests show that behaviours occur; they do not prove their absence elsewhere.

VII. DISCUSSION AND LIMITATIONS

Lessons. A renderer that turns every event into the line an older client expects made incremental growth cheap: twelve milestones later the terminal client still works, for plain conversations. Bounded queues cost a few milliseconds per message in our measurements but removed an entire class of failure, and writing the slow-receiver experiment forced us to say precisely what "isolation" means. Testing against real servers and the real window found problems that unit tests of pieces would not: malformed key data that raised an unhandled exception in a connection handler, a recovery-code "Done" button that was packed behind a sibling widget and so invisible, and an ordering bug in which a test thread waited on the server while the window loop it depended on was not running. Finally, end-to-end encryption is less about the cipher than about everything the server used to do with plaintext: search, pins, mentions, quoting, policy checks and moderation each needed a client-side or metadata-based replacement.

No forward secrecy. Private messages use an ephemeral key per message, which protects the sender's side, but theft of a recipient's long-term key would expose earlier messages to anyone holding the ciphertext, and the room's sealed chain copies stay on the server so that history can be re-read. A ratchet [31], [30] would be the upgrade, at the cost of storing per-device state.

Identity is as strong as the comparison. Keys are pinned on first sight and a change blocks sending, but until two people compare security codes a hostile server could have substituted a key at the very first contact. The same holds for the server certificate under trust on first use.

Metadata and moderation. The server learns who talks to whom and when, sizes and mentions. It cannot read encrypted content, so administrators cannot moderate by reading; they can delete by id, remove people and re-key.

Scope of encryption. Terminal clients and members without keys cannot take part in encrypted conversations; search of encrypted messages covers what the window has loaded; and files shared in readable conversations are stored unencrypted. The window is not a vault: decrypted text lives in memory while it is open.

Scale and evidence. The design is one server, one thread per client and one SQLite connection behind a lock. We do not establish where this stops being adequate; an `asyncio` server [36] under identical workloads, a multi-host study with packet loss and wireless links, and measurements with more clients are the obvious next experiments. The security claims rest on tests of this implementation, not on a formal analysis or an external review, and the primitives are audited but their composition here is new.

Future work. A ratchet for forward secrecy, multiple devices per person, sealed-sender style protection of metadata, an asynchronous server variant, and an independent security review.

VIII. CONCLUSION

LANChat-H shows that a chat system for a local network can combine a simple compatible protocol, isolation of slow clients, layered transport and account security, and end-to-end encryption for private conversations and an optional group room, in a code base small enough to read and with one optional dependency. The measurements on one machine show what the design costs and buys: a few milliseconds of extra latency per message in return for a room that survives a stalled receiver, and a fraction of a millisecond per encrypted message in return for a server that holds only ciphertext. A test that scans everything the server handles confirms that claim for the conversations tested. The limits are stated plainly: there is no forward secrecy, the server still sees metadata, no asynchronous variant was compared, and all measurements

come from one machine. The next steps are a ratchet, a multi-host evaluation, and an independent review.

REFERENCES

[1] J. Postel, "Transmission Control Protocol," IETF, RFC 793, Sep. 1981.

[2] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.

[3] R. Braden, Ed., "Requirements for Internet Hosts – Communication Layers," IETF, RFC 1122, Oct. 1989.

[4] V. Jacobson, "Congestion avoidance and control," in *Proc. ACM SIGCOMM*, 1988, pp. 314–329.

[5] M. Allman, V. Paxson, and E. Blanton, "TCP congestion control," IETF, RFC 5681, Sep. 2009.

[6] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Trans. Comput. Syst.*, vol. 2, no. 4, pp. 277–288, 1984.

[7] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Vol. 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003.

[8] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Pearson, 2011.

[9] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[10] F. Yergeau, "UTF-8, a transformation format of ISO 10646," IETF, RFC 3629, Nov. 2003.

[11] V. S. Pai, P. Druschel, and W. Zwaenepoel, "Flash: An efficient and portable Web server," in *Proc. USENIX Annu. Tech. Conf.*, 1999.

[12] M. Welsh, D. Culler, and E. Brewer, "SEDA: An architecture for well-conditioned, scalable Internet services," in *Proc. ACM SOSP*, 2001, pp. 230–243.

[13] J. K. Ousterhout, "Why threads are a bad idea (for most purposes)," invited talk, USENIX Winter Tech. Conf., 1996.

[14] R. von Behren, J. Condit, and E. Brewer, "Why events are a bad idea (for high-concurrency servers)," in *Proc. HotOS IX*, 2003.

[15] E. A. Lee, "The problem with threads," *Computer*, vol. 39, no. 5, pp. 33–42, May 2006.

[16] M. Krohn, E. Kohler, and M. F. Kaashoek, "Events can make sense," in *Proc. USENIX Annu. Tech. Conf.*, 2007.

[17] D. Kegel, "The C10K problem," 1999. [Online]. Available: http://www.kegel.com/c10k.html

[18] J. Oikarinen and D. Reed, "Internet Relay Chat protocol," IETF, RFC 1459, May 1993.

[19] C. Kalt, "Internet Relay Chat: Client protocol," IETF, RFC 2812, Apr. 2000.

[20] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Core," IETF, RFC 6120, Mar. 2011.

[21] P. Saint-Andre, "Extensible Messaging and Presence Protocol (XMPP): Instant messaging and presence," IETF, RFC 6121, Mar. 2011.

[22] I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, Dec. 2011.

[23] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019.

[24] S. Deering, "Host extensions for IP multicasting," IETF, RFC 1112, Aug. 1989.

[25] P. Th. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, Jun. 2003.

[26] K. P. Birman and T. A. Joseph, "Reliable communication in the presence of failures," *ACM Trans. Comput. Syst.*, vol. 5, no. 1, pp. 47–76, 1987.

[27] K. P. Birman, "The process group approach to reliable distributed computing," *Commun. ACM*, vol. 36, no. 12, pp. 37–53, Dec. 1993.

[28] E. Rescorla, "The Transport Layer Security (TLS) protocol version 1.3," IETF, RFC 8446, Aug. 2018.

[29] N. Unger *et al.*, "SoK: Secure messaging," in *Proc. IEEE Symp. Security and Privacy*, 2015, pp. 232–249.

[30] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila, "A formal security analysis of the Signal messaging protocol," in *Proc. IEEE Eur. Symp. Security and Privacy*, 2017, pp. 451–466.

[31] M. Marlinspike and T. Perrin, "The Double Ratchet algorithm," Signal, Rev. 1, Nov. 2016.

[32] Python Software Foundation, "socket," *Python 3 Documentation*. https://docs.python.org/3/library/socket.html

[33] Python Software Foundation, "threading," *Python 3 Documentation*. https://docs.python.org/3/library/threading.html

[34] Python Software Foundation, "struct," *Python 3 Documentation*. https://docs.python.org/3/library/struct.html

[35] Python Software Foundation, "ssl," *Python 3 Documentation*. https://docs.python.org/3/library/ssl.html

[36] Python Software Foundation, "asyncio," *Python 3 Documentation*. https://docs.python.org/3/library/asyncio.html

[37] J. Postel and J. Reynolds, "File Transfer Protocol (FTP)," IETF, RFC 959, Oct. 1985.

[38] T. Ylonen and C. Lonvick, Ed., "The Secure Shell (SSH) protocol architecture," IETF, RFC 4251, Jan. 2006.

[39] S. Cheshire and M. Krochmal, "Multicast DNS," IETF, RFC 6762, Feb. 2013.

[40] S. Cheshire and M. Krochmal, "DNS-based service discovery," IETF, RFC 6763, Feb. 2013.

[41] C. Percival and S. Josefsson, "The scrypt password-based key derivation function," IETF, RFC 7914, Aug. 2016.

[42] A. Langley, M. Hamburg, and S. Turner, "Elliptic curves for security," IETF, RFC 7748, Jan. 2016.

[43] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," IETF, RFC 8032, Jan. 2017.

[44] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF protocols," IETF, RFC 8439, Jun. 2018.

[45] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," IETF, RFC 5869, May 2010.

[46] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, "The Messaging Layer Security (MLS) protocol," IETF, RFC 9420, Jul. 2023.

[47] M. Marlinspike and T. Perrin, "The X3DH key agreement protocol," Signal, Rev. 1, Nov. 2016.

[48] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, Jun. 2017.

[49] B. Schneier and J. Kelsey, "Secure audit logs to support computer forensics," *ACM Trans. Inf. Syst. Secur.*, vol. 2, no. 2, pp. 159–176, May 1999.

[50] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proc. USENIX Security Symp.*, 2009, pp. 317–334.

[51] Python Software Foundation, "sqlite3," *Python 3 Documentation*. https://docs.python.org/3/library/sqlite3.html

[52] Python Software Foundation, "hashlib," *Python 3 Documentation*. https://docs.python.org/3/library/hashlib.html

[53] H. Goud B, R. R. Lingala, and S. Manocha, "LANChat-H: source code, tests and benchmark scripts," GitHub repository, 2026. [Online]. Available: https://github.com/revanthreddy0906/LAN-Chat-Application